QueryAPIGate DOCUMENTATION · PERFORMANCE

Performance on MySQL, v1: microservice databases, per-user queries

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE_MYSQL_V1

> **0.43's figures are in Performance on MySQL, v2,** which reruns this benchmark against 0.42 and answers this page's open questions. This page stays the description of the case and the method.

The case. One MySQL server holding six databases, one per microservice. Read-only queries look up one user at a time and join across those databases. No column masking. QueryAPIGate sits in front, with one connection and one read-only MySQL user.

The short answer. One gunicorn worker, as in the Docker image, with 32 concurrent callers:

QUERY (DATABASES IT JOINS)	MYSQL DIRECTLY, 32 THREADS	V0.41.0	`MAIN`	`MAIN` + METADATA CACHE
`customer` - one user's row (crm)	75,420/s	609	676	777
`order_status` - TPC-C Order-Status (crm, orders)	25,063/s	532	579	651
`order_history` - last orders with item names (crm, orders, catalog)	19,116/s	552	604	677
`payments` - the user's payments (crm, billing)	38,136/s	605	671	770
`profile` - the user with district and warehouse (crm, org)	50,026/s	609	677	772
`stock_level` - TPC-C Stock-Level (org, orders, inventory)	5,343/s	583	642	725

- **A call adds about 1.1-1.4 ms** to a query MySQL answers in 0.03-0.42 ms. That's the gateway's Python plus 3 round trips to MySQL. Every one of the 9.65 million requests returned 200.
- **One worker serves 580-780 of these a second.** The gateway, not MySQL, is the limit: MySQL could serve 8-110 times more, and at 8 workers it used at most 2.5 cores.
- **`main` is 9-11% faster than v0.41.0, and the metadata cache adds 12-15% more,** on every query. That's +22-28% in all.
- **Fewer threads are faster, with MySQL on the same host.** For a fast lookup, one thread per worker beat the Docker image's eight by 28%. Across a network, eight threads hid little of the extra waiting, for a reason found after this run: see A database across a network.
- **Skewed traffic plus a 30 s response cache:** when a few users are much busier than the rest (Zipf), 59% of calls came from the cache, throughput rose 23%, and MySQL got 58% fewer statements.

- **One security gap:** on MySQL, a key's table allow-list couldn't tell `crm.customer` from `billing.customer`. Fixed in 0.43. See Isolating services.

Since this run. 0.43 acts on what it found: a MySQL call makes 2 round trips instead of 3, and none of them stalls the worker (ADR 0027); table allow-lists are schema-qualified; `db._target()` is cached; the image's threads are a setting (`QUERYAPIGATE_THREADS`). These figures are 0.42's and `main`'s at the time. A v2 of this report measures 0.43.

How to read the numbers. Each cell is the median of 5 interleaved rounds, with its min-max range in the detailed tables. A difference counts as real only when two ranges don't overlap. The method is the one in Performance, v3, applied to MySQL.

Contents

- Method
- Results
- Where the time goes
- Isolating services
- Disk: the event log
- Sizing
- What to do with this
- Limits
- Run it yourself

Method

The machine

The same laptop as Performance, v3: an Intel i7-12700H, on AC power, with the `performance` power profile and energy preference. One session, 2026-10-24 19:33 to 2026-10-25 00:44 (5 h 11 min). Each part ran on its own cores:

PART	CPUS
QueryAPIGate (gunicorn)	0-7: four performance cores, both threads
MySQL 8.4.13	8-11: two performance cores
The load generator (oha), sysbench	12-19: the efficiency cores

The gateway reached MySQL at the container's own address, over TLS. mysql-connector negotiates TLS by default, and sysbench was run with `--mysql-ssl=REQUIRED`, so the direct numbers pay the same encryption cost.

A `/health` floor before and after every block stayed between 1,919 and 2,298 requests a second (−12% to +6% of its median), with the low end in the m6-m8 blocks. The CPU package ran at 66-100 °C. The load generator never used more than 0.63 of its 8 cores.

The builds

LABEL	COMMIT
v0.41.0	`f61d4ed`
v0.42.0	`647ab6f`
main	`3a92e5d`: 0.42.0 plus #95 (the store-file check skipped for server databases, the masking key cached, `QUERYAPIGATE_LOG_REQUESTS`)
main + cache	the same, with `QUERYAPIGATE_METADATA_CACHE_TTL=5`

Each runs as the Docker image does: gunicorn's `gthread` worker, 8 threads (the threads block below varies them, on `main`), the access log on. QueryAPIGate's own store is SQLite in its home directory - the single-instance default - except in the store comparison below.

Software

Python 3.12.3, gunicorn 26.2.0, mysql-connector-python 9.3.0 (with its C extension), MySQL 8.4.13 (`mysql:8.4`), oha pinned by digest (`ghcr.io/hatoo/oha@sha256:3ec3dbf549ea…`), go-tpc (the `latest` snapshot, commit `a9ca481`), and sysbench 1.1.0 (`perconalab/sysbench`). Linux 6.8.0. The full environment is the first line of `results.jsonl`.

The data: TPC-C, split into six databases

go-tpc loaded TPC-C at **20 warehouses**: 1.9 GB, which fits MySQL's 4 GB buffer pool. The data is TPC-C's, and two of the queries are TPC-C's read-only transactions, but this isn't the TPC-C benchmark, and its numbers aren't TPC-C results.

The nine tables were then moved into six databases, one per service, with `RENAME TABLE`:

DATABASE	TABLES	ROWS
`org`	`warehouse`, `district`	20, 200
`crm`	`customer`	600,000 - the users
`orders`	`orders`, `new_order`, `order_line`	600,000, 180,000, 6,000,000
`catalog`	`item`	100,000
`inventory`	`stock`	2,000,000
`billing`	`history` (payments)	600,000

TPC-C defines no index for looking a customer's payments up, so `billing.history` got one on its customer key, as the payments service would. The row counts are MySQL's estimates as recorded with the results.

QueryAPIGate has **one connection**, as user `qag`, which has `SELECT` on the six databases and nothing else. MySQL resolves `crm.customer JOIN orders.orders` within one session, so every cross-database join is one query.

The queries

Six saved queries. Each `user` query takes a customer by TPC-C's key (`w`, `d`, `c`). The full SQL is in `benchmarks/tpcc_mysql.py`.

QUERY	WHAT IT DOES	TABLES JOINED
`customer`	One customer's row by primary key	1
`order_status`	TPC-C Order-Status: the customer, their latest order and its lines	3
`order_history`	The customer's last orders with each item's name, 50 rows	4
`payments`	The customer's last 20 payments	2
`profile`	The customer with their district and warehouse	3
`stock_level`	TPC-C Stock-Level: distinct items below a stock threshold among a district's last 20 orders	3, a range and `COUNT(DISTINCT)`

One key, `reader`, may run all six; there are no masking rules.

The load

- **Customers are drawn from 600,000,** uniformly by default. The skew block draws them from a Zipf distribution (s = 1.0), so that a few customers are very busy. Ranks map through a fixed shuffle, so the hot customers are spread across warehouses.
- **Requests come from files of 200,000 URLs,** through oha's `--urls-from-file`. Because the files are generated rather than taken from a URL regex, the user ids really are uniform.
- **Closed loop** for throughput: 30 s runs after a 5 s warm-up. **Open loop** for latency under load, latency corrected, at 25-110% of capacity measured in the same block.
- **MySQL directly:** sysbench sends each query's SQL, with the same random parameters, in autocommit: one statement per call, as a service with its own connection would. sysbench's standard `oltp_read_only` runs on its own tables for reference.
- **What every run records:** throughput, latency percentiles, every status code; CPU for the gateway, MySQL and the load generator; MySQL's `Questions` counter, for statements per request (exact and immediate); and the clock and temperature.

Results

MySQL on its own

TEST	1 THREAD	32 THREADS
sysbench `oltp_read_only` (transactions/s)	1,695 (1,683–1,710)	4,425 (4,415–5,544)
`customer`	28,303/s, p50 0.03 ms	75,420/s
`order_status`	12,053/s, p50 0.08 ms	25,063/s
`order_history`	9,618/s, p50 0.10 ms	19,116/s
`payments`	18,479/s, p50 0.05 ms	38,136/s
`profile`	21,793/s, p50 0.04 ms	50,026/s
`stock_level`	2,373/s, p50 0.42 ms	5,343/s

Each `oltp_read_only` transaction is 16 statements. On two cores, MySQL answers these per-user joins in tens of microseconds. `stock_level`, with its range and `COUNT(DISTINCT)`, is the only one near half a millisecond.

One worker, 32 callers

QUERY	V0.41.0	V0.42.0	MAIN	MAIN + CACHE
`customer`	609 (609–615)	643 (641–649)	676 (672–679)	777 (772–778)
`order_status`	532 (529–537)	558 (555–560)	579 (577–582)	651 (642–652)
`order_history`	552 (550–553)	580 (579–583)	604 (601–607)	677 (675–678)
`payments`	605 (601–606)	638 (635–642)	671 (666–672)	770 (761–771)
`profile`	609 (607–610)	641 (639–642)	677 (676–679)	772 (768–777)
`stock_level`	583 (581–586)	613 (612–616)	642 (640–644)	725 (719–728)

All the ranges are clear of each other:

- **v0.42.0 over v0.41.0: +5–6%.** MySQL gets no help from 0.42's PostgreSQL-focused changes, but it does from reading the store less.
- **main over v0.42.0: +4–6%,** from #95: the store-file check is skipped for a server database like MySQL, and the masking key read is gone. Over v0.41.0, main is +9–11%.
- **The metadata cache adds +12–15% on top of main,** for +22–28% over v0.41.0 in all.
- **The gateway used 1.1–1.3 cores, and MySQL 0.08–0.47.** At p99, main is at 57–67 ms and the cache at 48–57 ms: that's 32 callers queueing for one worker, not slow queries.

The heavier the query, the less the gateway matters to it. `stock_level` runs at 12% of MySQL's direct rate through one worker, `customer` at 1%. A worker's ceiling is its Python, whatever the query.

One caller at a time

The p50 of a single call, with what the gateway adds to the same query sent directly:

QUERY	MYSQL DIRECTLY	V0.41.0	MAIN	MAIN + CACHE	THE GATEWAY ADDS (MAIN + CACHE)
`customer`	0.034 ms	1.28 ms	1.22 ms	1.16 ms	+1.12 ms
`order_status`	0.081 ms	1.54 ms	1.46 ms	1.40 ms	+1.32 ms
`stock_level`	0.417 ms	1.76 ms	1.70 ms	1.64 ms	+1.22 ms

- **A call costs about 1.1-1.4 ms more through the gateway,** whatever the query: a fixed cost, not a percentage.
- **p99 stays under 2.4 ms** in every build, and p99.9 under 3.3.
- **For one caller, v0.42.0 and v0.41.0 overlap** on two of the three queries. main is 3-5% faster than v0.41.0, and the cache another 3-5%.

Latency under load

`order_status`, open loop and latency-corrected. Capacity was 583 (581-584) calls a second for main and 652 (649-654) with the cache. Each cell is p50 / p99 / p99.9:

OFFERED	MAIN	MAIN + CACHE
25%	2.1 / 3.1 / 20.2 ms	2.0 / 2.9 / 12.3 ms
50%	1.8 / 2.5 / 10.5 ms	1.7 / 2.3 / 16.9 ms
75%	1.6 / 5.0 (3.0-8.8) / 29.8 ms	1.5 / 4.6 (3.0-12.1) / 21.2 ms
90%	1.6 / 5.5 (4.6-9.0) / 19.2 ms	1.7 / 9.0 (6.7-10.9) / 19.9 ms
100%	1.9 / 17.0 / 27.1 ms	1.9 / 17.8 / 24.6 ms
110%	450 / 857 / 864 ms; 619 of 640 served	597 / 1,366 / 1,383 ms; 673 of 716 served

- **p50 stays near 2 ms right up to capacity.** p99 is 3 ms to half load, about 5 ms at 75%, and 17 ms at 100%.
- **p99 varied more between rounds here than on PostgreSQL** (3.0-8.8 ms at 75%), so plan at 75%, as there.
- **p99.9 sits at 10-30 ms even at light load,** where PostgreSQL's was 6-15. It's a few slow calls in a thousand, not a trend.

More workers

64 callers:

WORKERS	`ORDER_STATUS` : MAIN	MAIN + CACHE	`STOCK_LEVEL` : MAIN	MAIN + CACHE	MYSQL CORES (`STOCK_LEVEL` , CACHE)
1	583 (581-585)	650 (647-654)	639 (636-642)	724 (721-726)	0.47
2	968 (963-972)	1,114 (1,111-1,116)	1,044 (1,041-1,048)	1,213 (1,207-1,219)	0.90
4	1,363 (1,357-1,378)	1,639 (1,633-1,646)	1,476 (1,471-1,485)	1,763 (1,758-1,774)	1.64
8	1,873 (1,805-1,936)	2,114 (2,092-2,202)	1,989 (1,914-2,032)	2,236 (2,213-2,254)	2.47

- **Throughput grows with workers, and the cache's lead holds at every size** (+11% to +20%).
- **Growth flattens past 2 workers.** Calls a second per gateway core fall from 469 at one worker to 262 at eight, for the reason v3 found: the eight workers share four physical cores' hyperthreads.
- **MySQL is far from its limit.** At 2,236 `stock_level` calls a second it used 2.5 of its 4 hardware threads, and for `order_status` 0.6.

Threads per worker

One worker of `main` , 32 callers, MySQL on the same machine:

THREADS	`CUSTOMER` (0.03 MS)	`STOCK_LEVEL` (0.42 MS)
1	**865 (864-879)**	630 (624-633)
2	823 (817-827)	**772 (769-777)**
4	722 (721-727)	683 (681-688)
8	676 (675-677)	642 (636-646)

All the ranges are clear. **The faster the query, the fewer threads it wants:**

- `customer` , at 0.03 ms in MySQL, is fastest with one thread: **28% faster than the default eight.**
- `stock_level` , at 0.42 ms, is fastest with two: 20% faster than eight. One thread is too few for it, because it then waits on MySQL with nothing else to do.

That fits the sizing rule below. Threads only help while calls spend real time waiting. This is with MySQL on the same host; a database across a network makes calls wait longer, which ought to favour more threads, but before 0.43 it didn't (next but one section).

Skewed users and the response cache

`order_status` at 32 callers. `order_status_cached` is the same query with `cache_ttl: 30` . Each run starts a fresh process, so the cache starts empty:

CUSTOMERS	QUERY	CALLS/S	P50	CACHE HITS	MYSQL STATEMENTS PER CALL
uniform	`order_status`	582 (578-583)	54.7 ms	-	2.00
uniform	`order_status_cached`	537 (535-541)	59.3 ms	5%	1.90
Zipf	`order_status`	581 (580-582)	54.7 ms	-	2.00
Zipf	`order_status_cached`	**717 (714-719)**	**44.3 ms**	**59%**	**0.83**

- **Skewed traffic is where a response cache pays.** With a few users making most of the calls, 59% were answered from the cache: +23% throughput, and 58% fewer statements reached MySQL.
- **Uniform traffic doesn't benefit.** With 600,000 equally likely users, only 5% of calls repeat within 30 s, and storing the 95% that never get reused costs 8%.
- **Skew alone changes nothing without the cache:** 581 against 582. MySQL's buffer pool already holds everything.
- **The response cache here is per process,** with no Redis: one worker, one cache. Several workers or instances without Redis each fill their own, so the hit rate falls; with `QUERYAPIGATE_REDIS_URL` they share one.

A database across a network

netem delays everything MySQL sends by 0.5 ms (about a same-zone cloud round trip). `order_status`:

	V0.41.0	V0.42.0	MAIN	MAIN + CACHE
One caller: p50	3.58 ms	3.58 ms	3.55 ms	3.39 ms
One caller: calls/s	280 (275-284)	279 (278-281)	283 (282-287)	293 (287-295)
32 callers: calls/s	421 (420-427)	436 (434-444)	450 (450-451)	489 (486-490)

- **A 0.5 ms hop adds about 2.1 ms to a call:** 3 round trips (see below), at 0.6 ms each under netem.
- **The builds barely differ for one caller.** All four make the same 3 round trips to MySQL, and the store, on local SQLite here, is off the network.
- **With 32 callers, throughput falls 22% (main),** from 579 to 450. Eight threads should have overlapped that waiting, and didn't.

Why, found after this run (2026-10-25): **mysql-connector's C extension holds Python's interpreter lock (GIL) while it waits for a ping,** and its `cursor()` pings on every call (see Round trips). So for one round trip in every call, the whole worker stops, not just the thread. A probe at the same 0.5 ms delay, calling mysql-connector directly from 1 and then 8 threads:

EACH THREAD, IN A LOOP	1 THREAD	8 THREADS	8 THREADS ÷ 1
`SELECT 1`	1,594/s	12,733/s	8.0×
`ROLLBACK`	1,700/s	13,342/s	7.8×
`ping()`	1,703/s	1,617/s	**0.9×**
What the gateway did per call: `cursor()`, the query, `ROLLBACK`	541/s	1,391/s	**2.6×**

Statements overlap across threads, as they should; pings don't overlap at all. TLS wasn't the cause: `SELECT` scaled 8.3× with it and 8.2× without. 0.43 keeps one cursor per pooled connection, so there's no ping per call (ADR 0027).

The store across the network

Here the network matters. QueryAPIGate's own store on local SQLite, against PostgreSQL across the same 0.5 ms hop, which is what several instances behind a load balancer share. One caller, `order_status`:

STORE	MAIN	MAIN + CACHE
SQLite, local	3.52 ms (284/s)	3.39 ms (293/s)
PostgreSQL, 0.5 ms away	**6.06 ms (166/s)**	**3.43 ms (292/s)**

- **Without the cache, a store across the network costs 2.5 ms a call,** the 3 reads it makes, and cuts single-caller throughput by 42%.
- **With the cache, it costs nothing:** 3.43 ms against 3.39 ms local. For a multi-instance deployment on a shared PostgreSQL store, the metadata cache is the difference between those two rows.

Where the time goes

Round trips

Counted with MySQL's general log, 100 calls after a warm-up, in every build:

STATEMENTS PER CALL	
The query	1
`ROLLBACK`, when the connection goes back to the pool	1
Total, every build	**2**

That's half of PostgreSQL's 4. The MySQL driver makes a connection read-only once, when it opens it (`SET SESSION TRANSACTION READ ONLY`), and sets the time limit only when it changes. So no per-query `BEGIN` or `SET` is needed. (That turned out to be unsafe: a stored function called as `SELECT f()` could change both for later calls. 0.43 makes the connection read-only per transaction and puts the time limit on each statement - ADR 0027, and the CHANGELOG's Security entry.)

Measured, there are 3. Single-call p50 at 0, 0.5, 1 and 2 ms of netem delay; the slope is round trips per call. mysql-connector on its own, sending a known 1 or 2 statements, calibrates it:

	+0 MS	+0.5 MS	+1 MS	+2 MS	SLOPE	ROUND TRIPS (SLOPE ÷ 1.2)
mysql-connector, 1 statement	0.04	0.64	1.23	2.49	1.22	1
mysql-connector, query + `ROLLBACK`	0.05	1.26	2.39	4.75	2.35	2.0
v0.41.0	1.27	3.30	4.94	8.57	3.64	3.0
v0.42.0	1.27	3.28	4.91	8.68	3.59	3.0
main	1.21	3.21	4.86	8.25	3.53	2.9
main + cache	1.16	3.06	4.78	8.48	3.66	3.1

The third round trip isn't a statement, which is why the log doesn't show it. **mysql-connector's `cursor()` pings the server** (`is_connected()` → `COM_PING`) every time it's called, and QueryAPIGate's MySQL driver makes a new cursor for every query. Reusing one cursor per pooled connection would save one round trip in three: about 0.6 ms a call across a 0.5 ms network - and, as A database across a network shows, a stall of the whole worker. 0.43 does this.

No connections were opened mid-run. MySQL's `Connections` counter didn't move during 10 s of load at 1, 8 or 32 callers, so this isn't reconnecting; the pool holds.

The CPU

py-spy, 200 samples a second, during 30 s of `order_status` at 32 callers. Each sample is counted under the innermost frame that falls in a category:

WHERE THE SAMPLES FALL	MAIN	MAIN + CACHE
Store layer: the key, the query and the connection, read from SQLite	25.2%	5.7%
Data query: mysql-connector and the pool	20.7%	24.9%
Logging	14.0%	17.3%
gunicorn, sockets, HTTP	13.9%	17.7%
Flask and Werkzeug	6.7%	9.2%
History writer thread	5.9%	6.0%
JSON	4.0%	6.5%
SQL checks, parameters	3.7%	5.7%
QueryAPIGate's request handling	3.2%	4.2%
Authentication, limits, quotas, metrics	2.2%	2.3%
Other	0.6%	0.5%

What the busiest frames show:

- **Even a local SQLite store shows as a fifth of the samples.** Reading the key, the saved query and the connection is 7.3% + 7.3% + 6.3%: Python's `sqlite3` and JSON decoding, not I/O. The metadata cache takes it to 5.7%. That overstates what the cache saves: py-spy counts a thread inside `sqlite3` even while it has released the GIL and another thread is running Python, so these samples aren't all CPU the worker could have used. The measured gain is 12–15%.
- **`db._target()` takes 3.6–3.8%,** with or without the cache. It re-reads the store's settings from the environment on every call; caching it is a small, safe fix.
- **mysql-connector's ping on every `cursor()`** shows as 2.6–3.8%, and its `ROLLBACK` as 3.1–3.6%.
- **Threads waiting on the logging module's lock: 5–9%,** as on PostgreSQL. `QUERYAPIGATE_LOG_REQUESTS=off` (in main) removes most of it.

Isolating services

Six services in one MySQL server raise a question PostgreSQL's case didn't: can a key for one service be kept out of another's tables? This is the result of checking, not a performance number. A key with `"allowed_tables": ["customer"]`, ad-hoc SQL, and a decoy `billing.customer` table:

SQL	RESULT
`SELECT COUNT(*) FROM crm.customer`	200: allowed, as intended
`SELECT COUNT(*) FROM billing.customer`	**200: allowed, though it's another service's table**
`SELECT COUNT(*) FROM billing.history`	403 `table_not_allowed`, as intended

On MySQL, the table allow-list matched bare table names (0.42 and earlier), so `customer` meant `customer` in any database the connection's user could reach. Only Trino connections matched `catalog.schema.table`.

Fixed in 0.43: grants are schema-qualified on every SQL connection, so `customer` means the connection's own database's `customer`, and `billing.customer` is refused unless granted (the CHANGELOG's Security entry). Keeping services apart with MySQL's own grants is still the stronger boundary: one MySQL user per service, granted only its databases, each as its own QueryAPIGate connection, with keys granted only their service's connection. The integration tests check both.

Saved queries aren't affected. A key granted named queries can only run those queries' SQL, and that SQL is written by whoever publishes it.

Disk: the event log

Every call is recorded in the store's event log (ADR 0020), which keeps events for 24 hours by default. In this run, the single-instance SQLite store took in 10.6 million events in five hours: **6 GB, about 570 bytes a call.** That's more events than the 9.65 million requests in this report's totals, because the totals leave out the warm-ups before each run and the calls the round-trip block (m9) made to count statements. Run history, by contrast, stays small, because it keeps only the newest 50 runs per saved query version.

This is the store's event log, in `queryapigate.db`. The server's own log on stdout is separate, and grew to 6.6 GB in the same run (see What to do with this).

So plan the store's disk for the traffic you serve:

SUSTAINED CALLS A SECOND	EVENT LOG AFTER 24 HOURS
50	about 2.5 GB
500	about 25 GB
2,000	about 100 GB

`QUERYAPIGATE_EVENTS_RETENTION_HOURS` shortens how long events are kept, and the disk needed shrinks with it. On a PostgreSQL store, rows are larger, so expect more.

Sizing

The rule from Performance, v3, with MySQL's figures:

- **CPU:** plan on about 440 calls a second per worker for `order_status`-class queries, or 490 with the metadata cache. That's 75% of measured capacity, where p99 is around 5 ms. Single-table lookups go about 15% higher.
- **Waiting:** a worker serves at most threads ÷ (query time + 3 round trips × network round-trip time) calls a second.
- **Workers needed** = the larger of the two figures divided into your calls a second, plus one spare.

Two examples, with a 0.5 ms network round trip:

CALLS	CPU FIGURE	WAITING FIGURE	THREADS TO USE
A per-user lookup, 0.1 ms in MySQL	440 a second	8 ÷ (0.1 + 3 × 0.5 ms) = 5,000 a second	CPU-bound: 1-2 threads beat 8
A 20 ms report query	440 a second, or fewer	8 ÷ (20 + 1.5 ms) = 370 a second	Waiting-bound: more threads help

MySQL won't be what you size first. On two cores it ran these queries 8-110 times faster than one gateway worker could ask for them. Its limit shows only with many workers on the heavier queries: 2.5 cores at 8 workers on `stock_level`.

What to do with this

For running QueryAPIGate in front of MySQL:

- **One connection with a read-only user covering the services' databases** works, and joins across them cost no more than joins within one. Use one connection per service if the services must be kept apart (see Isolating services).
- **Turn on the metadata cache:** +12-15% on one instance, even with the store on local SQLite. With several instances on a shared PostgreSQL store, it removes 2.5 ms of store round trips from every call. Its trade-off - a change made on another instance takes up to the TTL to arrive - is explained in PERFORMANCE.md.
- **For per-user lookups on a database on the same host, run 1-2 threads per worker,** not 8: up to 28% more throughput. Keep more for slow queries, and - from 0.43 - for a database across a network. Set them with `QUERYAPIGATE_THREADS` (from 0.43; see Tune a deployment for throughput).
- **Put `cache_ttl` on queries whose callers repeat themselves** - a dashboard a user keeps refreshing, popular accounts. Don't put it on queries every user calls once: there it costs 8%.

- **Turn off the per-request log lines** if a proxy already logs requests: `QUERYAPIGATE_LOG_REQUESTS=off`, and from 0.43 `QUERYAPIGATE_ACCESS_LOG=off` for gunicorn's access log. At these rates the server's log on stdout grows by gigabytes an hour: 6.6 GB in this five-hour run, apart from the store's event log below.
- **Size the store's disk for the event log:** about 570 bytes a call, kept for 24 hours (see Disk).

For QueryAPIGate itself - new from this run:

1. **Reuse one cursor per pooled MySQL connection.** mysql-connector pings the server on every `cursor()`; that's 1 of the 3 round trips a call makes. *Done in 0.43 (ADR 0027).*
2. **Match `schema.table` in MySQL table allow-lists,** as Trino already does. That's a security fix for the multi-service case, not a performance one. *Done in 0.43, on every dialect.*
3. **Cache `db._target()`:** 3.6-3.8% of the CPU, re-reading environment settings that don't change. *Done in 0.43.*
4. **Read-only MySQL connections could autocommit,** which would drop the `ROLLBACK` round trip too. The session is already read-only, so it's likely safe, but stored functions need checking first. This belongs in the same design note as the PostgreSQL round trips (ADR 0027). *Checked, and not safe: a stored function can make the session read-write. 0.43 chains a read-only transaction instead (`ROLLBACK AND CHAIN`), still 2 round trips.*

Limits

- **One machine,** on separate cores. The scale-out is workers sharing four physical cores, not separate hosts.
- **Every query is cheap,** because the 1.9 GB of data sits in MySQL's buffer pool. Real per-user queries that touch disk, or take tens of milliseconds, move the limit from the gateway's CPU to waiting. The sizing rule covers that, but this run didn't measure it.
- **The network is simulated:** one-way delay, no jitter or loss. No TLS termination, load balancer or Redis in front of the gateway.
- **One query shape per run.** Real services mix them. Mixing reads and writes was measured on PostgreSQL (v3), not here. Writes weren't measured on MySQL at all.
- **MySQL's general log counts statements, not packets.** The third round trip was found by timing, and explained by reading mysql-connector's code.
- **p99.99** wasn't measured: there were no 150 s runs in this session.

Run it yourself

You need Docker, `taskset`, the go-tpc binary, and QueryAPIGate's server, MySQL and PostgreSQL dependencies. py-spy is optional:

```shell
pip install -e ".[server,mysql,postgres]" py-spy
python benchmarks/tpcc_mysql.py --out /tmp/qag-tpcc --go-tpc ./go-tpc setup A B teardown   # about 5.5 hours
python benchmarks/tpcc_mysql_report.py /tmp/qag-tpcc/results.jsonl
```

- **`--out` must be an absolute path.**
- **`setup`** starts MySQL 8.4, loads 20 warehouses (`--warehouses`) in under a minute, moves the tables into the six databases and creates the read-only user. **`A`** runs the direct baseline, throughput and single-call blocks; **`B`** runs the rest.

- `--refs` chooses the builds (v0.41.0, v0.42.0 and `origin/main` by default; the last also runs with the metadata cache), and `--gateway-cpus`, `--db-cpus` and `--load-cpus` the cores.
- `--smoke` runs every block for 2 s to check the setup.
- `teardown` removes the containers and their data volumes.
- **Plug the laptop in first,** and check `powerprofilesctl get`.

The 2026-10-24 run is in `benchmarks/results/tpcc-mysql-2026-10-24/`: one JSON line per run, every table, and the py-spy profiles.

READ NEXT
Performance, v2: an OLTP benchmark on PostgreSQL

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE_MYSQL_V1

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit
@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate